

¿Qué es Haskell?



Haskell Brooks Curry



- **GHC:** GHC (Glasgow Haskell Compiler) es el compilador principal para el lenguaje de programación Haskell. Es una implementación de Haskell que se utiliza para traducir el código fuente escrito en Haskell a código de máquina ejecutable.

¿Cómo instalar Haskell?

Entonces, ¿qué es Haskell?

Haskell es un **lenguaje de programación puramente funcional**. En los lenguajes imperativos, se hacen las cosas dándole a la computadora una secuencia de tareas y luego ésta las ejecuta. Mientras los ejecuta, puede cambiar de estado. Por ejemplo, configura la variable `a` en 5 y luego hace algunas cosas y luego la configura en otra cosa. Tiene estructuras de flujo de control para realizar alguna acción varias veces. En la programación puramente funcional, no le dices a la computadora qué hacer, sino más bien qué cosas *son*. El factorial de un número es el producto de todos los números desde 1 hasta ese número, la suma de una lista de números es el primer número más la suma de todos los demás números, y así sucesivamente. Lo expresas en forma de funciones. Tampoco puedes establecer una variable para algo y luego establecerla para otra cosa más tarde. Si dices que `a` es 5, no podrás decir que es otra cosa después porque acabas de decir que era 5. ¿Qué eres, una especie de mentiroso? Entonces, en lenguajes puramente funcionales, una función no tiene efectos secundarios. Lo único que puede hacer una función es calcular algo y devolverlo como resultado. Al principio, esto parece un poco limitante, pero en realidad tiene algunas consecuencias muy interesantes: si se llama a una función dos veces con los mismos parámetros, se garantiza que devolverá el mismo resultado. Eso se llama transparencia referencial y no solo permite al compilador razonar sobre el comportamiento del programa, sino que también le permite deducir fácilmente (e incluso probar) que una función es correcta y luego construir funciones más complejas uniendo funciones simples.



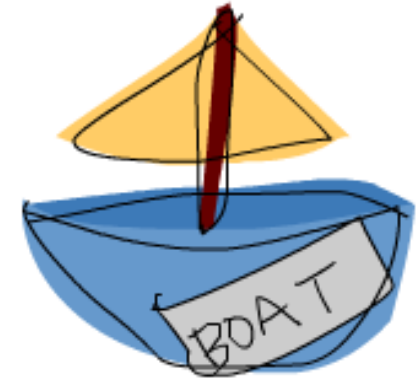
Haskell es **un vago** . Eso significa que, a menos que se indique específicamente lo contrario, Haskell no ejecutará funciones ni calculará cosas hasta que se vea realmente obligado a mostrarle un resultado. Eso va bien con la transparencia referencial y permite pensar en los programas como una serie de **transformaciones de datos** . También permite cosas interesantes como estructuras de datos infinitas. Digamos que tienes una lista inmutable de números `xs = [1,2,3,4,5,6,7,8]` y una función `doubleMe` que multiplica cada elemento por 2 y luego devuelve una nueva lista. Si quisiéramos multiplicar nuestra lista por 8 en un lenguaje imperativo y lo hiciéramos `doubleMe(doubleMe(doubleMe(xs)))` , probablemente pasaría por la lista una vez, haría una copia y luego la devolvería. Luego pasaría por la lista otras dos veces y devolvería el resultado. En un lenguaje perezoso, llamar a `doubleMe` en una lista sin forzarlo a mostrarte el resultado termina en el programa diciéndote "¡Sí, sí, lo haré más tarde!". Pero una vez que quieres ver el resultado, el primer `doubleMe` le dice al segundo que quiere el resultado, ¡ahora! El segundo le dice eso al tercero y el tercero de mala gana le devuelve un 1 duplicado, que es un 2. El segundo recibe eso y le devuelve 4 al primero. El primero ve eso y le dice que el primer elemento es 8. Por lo tanto, solo pasa una vez por la lista y solo cuando realmente lo necesita. De esa manera, cuando quieras algo de un lenguaje vago, puedes simplemente tomar algunos datos iniciales y transformarlos y corregirlos de manera eficiente para que se parezca a lo que deseas al final.



Haskell está **tipado estáticamente** . Cuando compila su programa, el compilador sabe qué fragmento de código es un número, cuál es una cadena, etc. Eso significa que se detectan muchos errores posibles en el momento de la compilación. Si intenta sumar un número y una cadena, el compilador se quejará. Haskell utiliza un muy buen sistema de tipos que tiene **inferencia de tipos** . Eso significa que no es necesario etiquetar explícitamente cada fragmento de código con un tipo porque el sistema de tipos puede descubrir mucho sobre él de manera inteligente. Si dices `a = 5 + 4` , no tienes que decirle a Haskell que `a` es un número, él puede descubrirlo por sí solo.

La inferencia de tipos también permite que su código sea más general. Si una función que crea toma dos parámetros y los suma y no indica explícitamente su tipo, la función funcionará en dos parámetros cualesquiera que actúen como números.

Haskell es **elegante y conciso** . Debido a que utiliza muchos conceptos de alto nivel, los programas de Haskell suelen ser más cortos que sus equivalentes imperativos. Y los programas más cortos son más fáciles de mantener que los más largos y tienen menos errores.



Lo que necesitas para sumergirte

Un editor de texto y un compilador Haskell. Probablemente ya tengas instalado tu editor de texto favorito, así que no perderemos el tiempo en eso. Para los propósitos de este tutorial usaremos GHC, el compilador Haskell más utilizado. La mejor manera de comenzar es descargar la [Plataforma Haskell](#), que es básicamente Haskell con baterías incluidas.

GHC puede tomar un script de Haskell (normalmente tienen una extensión `.hs`) y compilarlo, pero también tiene un modo interactivo que le permite interactuar de forma interactiva con los scripts. Interactivamente. Puede llamar funciones desde scripts que cargue y los resultados se muestran inmediatamente. Para aprender, es mucho más fácil y rápido que compilar cada vez que realiza un cambio y luego ejecutar el programa desde el indicador. El modo interactivo se invoca escribiendo `ghci` cuando se le solicite. Si ha definido algunas funciones en un archivo llamado, digamos, `myfunctions.hs`, cargue esas funciones escribiendo `:l myfunctions` y luego podrá jugar con ellas, siempre que `myfunctions.hs` esté en la misma carpeta desde la que se invocó `ghci`. Si cambia el script `.hs`, simplemente ejecute `:l myfunctions` nuevamente o haga `:r`, que es equivalente porque recarga el script actual. El flujo de trabajo habitual para mí cuando juego con cosas es definir algunas funciones en un archivo `.hs`, cargarlo y jugar con ellas y luego cambiar el archivo `.hs`, cargarlo nuevamente y así sucesivamente. Esto es también lo que haremos aquí.

Empezando

¡En sus marcas, listos, fuera!

Muy bien, ¡comencemos! Si eres el tipo de persona horrible que no lee las introducciones a las cosas y te las saltaste, quizás quieras leer la última sección de la introducción de todos modos porque explica lo que necesitas para seguir este tutorial y cómo vamos. para cargar funciones. Lo primero que haremos es ejecutar el modo interactivo de ghc y llamar a alguna función para tener una idea muy básica de Haskell. Abra su terminal y escriba `ghci` . Serás recibido con algo como esto.

```
GHCI , versión 6.8 . 2 : http://www.haskell.org/ghc/ :? por ayuda
Cargando base de paquetes... vinculando... listo.
Preludio >
```



¡Felicitaciones, estás en GHCi! El mensaje aquí es `Prelude>` pero debido a que puede alargarse cuando cargas cosas en la sesión, usaremos `ghci>` . Si desea tener el mismo mensaje, simplemente escriba `:set Prompt "ghci>"` .

Las funciones generalmente tienen prefijo, por lo que de ahora en adelante no indicaremos explícitamente que una función tiene la forma de prefijo, simplemente lo asumiremos. En la mayoría de los lenguajes imperativos, las funciones se llaman escribiendo el nombre de la función y luego sus parámetros entre paréntesis, generalmente separados por comas. En Haskell las funciones se llaman escribiendo el nombre de la función, un espacio y luego los parámetros, separados por espacios. Para empezar, intentaremos llamar a una de las funciones más aburridas de Haskell.



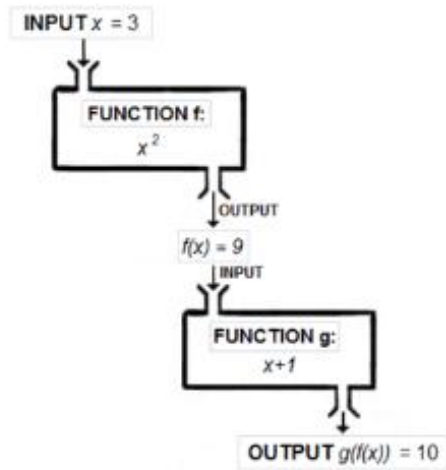
```
ghci> succ 8
9
```

La función `succ` toma cualquier cosa que tenga un sucesor definido y devuelve ese sucesor. Como puede ver, simplemente separamos el nombre de la función del parámetro con un espacio. Llamar a una función con varios parámetros también es sencillo. Las funciones `min` y `max` toman dos cosas que se pueden ordenar (¡como números!). `min` devuelve el que es menor y `max` devuelve el que es mayor. Ver por ti mismo:

```
ghci> min 9 10
9
ghci> min 3.4 3.2
3.2
ghci> max 100 101
101
```

La aplicación de funciones (llamar a una función poniendo un espacio después de ella y luego escribiendo los parámetros) tiene la mayor prioridad de todas. Lo que eso significa para nosotros es que estas dos afirmaciones son equivalentes.

```
ghci> succ 9 + max 5 4 + 1
16
ghci> (succ 9) + (max 5 4) + 1
16
```

-- Función para la suma

suma :: Int -> Int -> Int

suma x y = x + y

-- Función para la resta

resta :: Int -> Int -> Int

resta x y = x - y

-- Función para la multiplicación

multiplicacion :: Int -> Int -> Int

multiplicacion x y = x * y

-- Ejemplos de uso

ejemploSuma :: Int

ejemploSuma = suma 5 3 -- Resultado: 8

ejemploResta :: Int

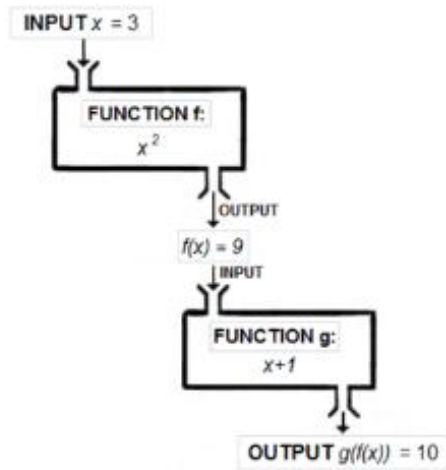
ejemploResta = resta 10 4 -- Resultado: 6

ejemploMultiplicacion :: Int

ejemploMultiplicacion = multiplicacion 3 7 -- Resultado: 21

ejemploDivision :: Int

ejemploDivision = division 20 5 -- Resultado: 4



-- Función segura para la división

`divisionSegura :: Int -> Int -> Maybe Int`

`divisionSegura x 0 = Nothing` -- División por cero, devolvemos Nothing

`divisionSegura x y = Just (x `div` y)`

-- Ejemplos de uso

`ejemploDivisionSegura1 :: Maybe Int`

`ejemploDivisionSegura1 = divisionSegura 20 5` -- Resultado: Just 4

`ejemploDivisionSegura2 :: Maybe Int`

`ejemploDivisionSegura2 = divisionSegura 10 0` -- Resultado: Nothing

Funciones - Functions

¿Qué es exactamente una función? Esta es una pregunta importante para hacer y entender si usted vas a explorar la programación funcional. El comportamiento de las funciones en Haskell viene directamente de las matemáticas. En matemáticas, a menudo decimos cosas como $f(x) = y$, lo que significa que hay alguna función f que toma un parámetro x y mapea a un valor y . En matemáticas, **if $f(2) = 2.000.000$** para una función f dada, nunca puede darse el caso de que **$f(2) = 2.000.001$** . El lector atento se preguntará: **«¿Y la función raíz cuadrada? 4 tiene dos raíces, 2 y -2, así que ¿cómo puede ser $\text{sqrt } x$ una función verdadera cuando apunta claramente a dos y s?»**. La clave es que x e y no tienen por qué ser lo mismo. Podemos decir que $\text{sqrt } x$ es la raíz positiva, por lo que tanto x como y son números reales positivos, lo que resuelve este problema. Pero también podemos tener $\text{sqrt } x$ ser una función de un número real positivo a pares de números reales. En este caso, cada x corresponde exactamente a un par. En Haskell, las funciones funcionan exactamente igual que en matemáticas. La figura 2.1 muestra una función llamada **simple**.

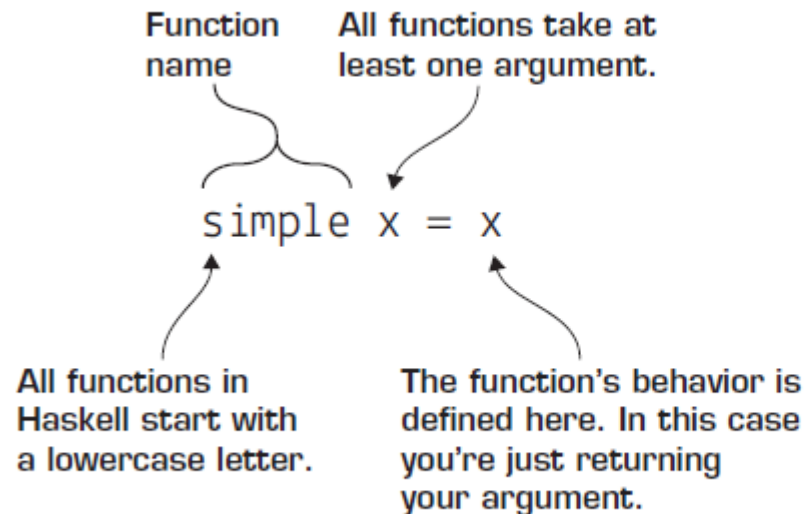


Figure 2.1 Defining a simple function

La función simple toma un único argumento x y lo devuelve intacto. Observa que, a diferencia de muchos otros lenguajes de programación, en Haskell no necesitas especificar que estás devolviendo un valor. En Haskell, las funciones deben devolver un valor, así que nunca hay necesidad de hacer esto explícito. Puede cargar su función simple en GHCi y ver cómo se comporta. Para cargar una función, todo lo que tienes que hacer es tenerla en un fichero y usar **:load <nombre de fichero>** en GHCi:

```
GHCi> simple^2
```

```
2
```

```
GHCi> simple "dog"
```

```
"dog"
```

Todas las funciones en Haskell siguen tres reglas que las obligan a comportarse como funciones en matemáticas:

1. Todas las funciones deben tomar un argumento.
2. Todas las funciones deben devolver un valor.
3. Cada vez que una función es llamada con el mismo argumento, debe devolver el mismo valor.

La tercera regla forma parte de la definición matemática básica de una función. Cuando la regla que el mismo argumento debe producir siempre el mismo resultado se aplica a una función en un lenguaje de programación, se llama **transparencia referencial**.

Listas en Haskell

Para Haskell (y la programación funcional en general), la estructura de datos fundamental es una lista. Incluso cuando te acerques a algunos de los temas más avanzados de este libro, como funtores y mónadas, la lista simple seguirá siendo el ejemplo más útil.

Las listas son la estructura de datos más importante de la programación funcional. Una de razones es que las listas son intrínsecamente recursivas. Una lista puede ser una lista vacía o un elemento seguido de otra lista. Desmontar y construir listas son herramientas fundamentales para muchas técnicas de programación funcional. Al desmontar una lista, las piezas principales son la cabeza, la cola y el final (representado por []). La cabeza es el primer elemento de una lista:

```
GHCi> head [1,2,3]
1
GHCi> head [[1,2],[3,4],[5,6]]
[1,2]
```

La cola es el resto de la lista que queda después de la cabeza:

```
GHCi> tail [1,2,3]
[2,3]
GHCi> tail [3]
[]
```


Puedes visualizar esto imaginando que arrancas el primer de una lista de la compra, como en la figura 6.1.

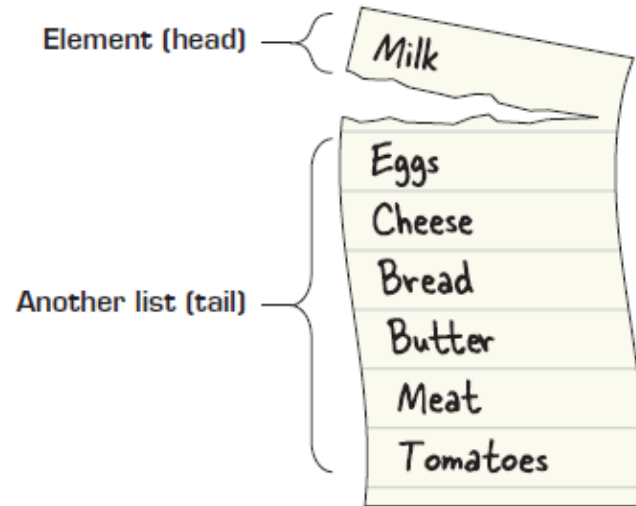


Figure 6.1 A list is made up of the head element and the tail list.

Puedes dividir una lista en trozos, pero de poco te servirá si no puedes volver a unirlos. En programación funcional, construir listas es tan importante como descomponerlas. Para construir una lista, sólo necesitas una función y el operador infijo (:), que se llama cons. Este término es la abreviatura de construct y tiene su origen en Lisp. Nos referiremos a esta operación como consing, porque : queda un poco raro en una frase.

Para hacer una lista, hay que tomar un valor y conjugarlo con otra lista. La forma más sencilla de hacer una lista es juntar un valor con la lista vacía:

```
GHCi> 1:[]  
[1]
```

Bajo el capó, todas las listas en Haskell se representan como un montón de operaciones consing,y la notación [...] es azúcar sintáctico (una característica de la sintaxis del lenguaje de programación diseñado únicamente para hacer las cosas más fáciles de leer):

```
GHCi> 1:2:3:4:[]  
[1,2,3,4]
```

```
GHCi> (1,2):(3,4):(5,6):[]  
[(1,2),(3,4),(5,6)]
```

Observa que todas estas listas terminan con la lista vacía []. Por definición, una lista es siempre un con otra lista (que también puede ser una lista vacía).

Podría adjuntar el valor al principio de una lista existente si lo desea:

```
GHCi> 1:[2,3,4]  
[1,2,3,4]
```

Vale la pena señalar que las cadenas que has visto hasta ahora son en sí mismas azúcar sintáctico para listas de caracteres (entre comillas simples en lugar de dobles):

```
GHCi> ['h','e','l','l','o']  
"hello"  
GHCi> 'h': 'e': 'l': 'l': 'o': []  
"hello"
```

Una cosa importante a recordar es que en Haskell cada elemento de la lista debe ser del mismo tipo. Por ejemplo, puede **cons** la letra 'h' a la cadena «ello» porque «ello» es una lista de caracteres y 'h' (comillas simples) es un carácter:

```
GHCi> 'h': "ello"  
"hello"
```

Pero no puede contraponer «h» (comillas dobles) a «ello» porque «h» es una lista de un carácter y los valores dentro de «ello» son caracteres individuales. Esto se hace más evidente el azúcar sintáctico.

Listing 6.1 Consing characters and strings

```
GHCi> "h":"ello"           ← Error!
GHCi> ['h']:['e','l','l','o'] ← Same code with one layer
                                of sugar removed
GHCi> 'h':[]:'e': 'l': 'l': 'o':[] ← Completely desugared
```

Si quieres combinar dos listas, tienes que concatenarlas utilizando `++`. La concatenación de texto, pero dado que las cadenas son sólo listas, funcionará con cualquier lista: `++`.

```
GHCi> "h" ++ "ello"
"hello"
GHCi> [1] ++ [2,3,4]
[1,2,3,4]
```

Consing es importante de entender porque es una parte esencial de la escritura recursiva sobre listas. Casi todas las operaciones secuenciales en programación funcional implican construir listas, descomponerlas o una combinación de ambas.

Listas y evaluación perezosa | Lists and lazy evaluation

Debido a que las listas son tan importantes en Haskell, hay muchas maneras de generar rápidamente rangos de datos. He aquí algunos ejemplos:

```
GHCi> [1 .. 10]
[1,2,3,4,5,6,7,8,9,10]
```

← Generates a list of numbers from 1 through 10

```
GHCi> [1,3 .. 10]
[1,3,5,7,9]
```

← Adding the next step, 3, generates odd numbers.

```
GHCi> [1, 1.5 .. 5]
[1.0,1.5,2.0,2.5,3.0,3.5,4.0,4.5,5.0]
```

← Generates a list in increments of 0.5

```
GHCi> [1,0 .. -10]
[1,0,-1,-2,-3,-4,-5,-6,-7,-8,-9,-10]
```

← Generates a decrementing list

Son útiles, pero no especialmente interesantes. Muchos lenguajes de programación tienen una función que funciona de forma similar. ¿Qué ocurre si se olvida de poner un límite superior a tu rango?

```
GHCi> [1 .. ]
[1,2,3,4,5,6,7,8,9,10,11,12 .. ]
```

Se genera una lista interminable. Esto está bien, pero atasca rápidamente el terminal y no parece especialmente útil. Lo interesante es que puedes asignar esta lista a una variable e incluso utilizarla en una función:

```
simple x = x
longList = [1 .. ]
stillLongList = simple longList
```

Funciones comunes sobre listas | Common functions on lists

Dado que las listas son tan importantes, se ha incorporado una amplia gama de funciones útiles en el módulo de la biblioteca estándar de Haskell, llamado Prelude. Hasta ahora, has visto `head`, `tail`, `:` y `++`, que te permiten desarmar y armar listas. Hay muchas otras funciones útiles sobre listas que aparecerán con tanta frecuencia al escribir Haskell que vale la pena familiarizarse con ellas.

El operador `!!`

Si quieres acceder a un elemento concreto de una lista por su índice, puedes utilizar el operador `!!`. El operador `!!` toma una lista y un número, y devuelve el elemento en esa posición de la lista. Las listas en Haskell están indexadas empezando en 0. Si intentas acceder a un valor más allá del final de la lista, no podrás hacerlo, obtendrá un error:

```
GHCI> [1,2,3] !! 0
```

```
1
```

```
GHCI> "puppies" !! 4
```

```
'i'
```

```
GHCI> [1..10] !! 11
```

```
*** Exception: Prelude.!!: index too large
```

Como se mencionó, cualquier operador infijo (un operador que se coloca entre dos valores, como `+`) también puede usarse como una función prefija encerrándolo entre paréntesis:

```
GHCI> (!! ) [1,2,3] 0
```

```
1
```

El uso de la notación de prefijos a menudo puede facilitar cosas como la aplicación parcial. Prefijo también es útil para utilizar operadores como argumentos de otras funciones. Puede seguir utilizando

utilizar la aplicación parcial con un operador infijo; basta con envolver la expresión entre paréntesis:

```
GHCi> paExample1 = (!! ) "dog"
GHCi> paExample1 2
'g'
GHCi> paExample2 = ("dog" !! )
GHCi> paExample2 2
'g'
```

Fíjate que en paEjemplo2 ves cómo funciona la aplicación parcial con operadores binarios infijos. Para realizar la aplicación parcial en un operador binario, llamado sección, necesita envolver la expresión entre paréntesis. Si sólo incluye el argumento de la derecha, la función esperará al argumento de más a la izquierda; si incluye sólo el argumento de la izquierda, obtendrá una función esperando el argumento de la derecha. Este es el paEjemplo3,

Que crea la aplicación parcial del argumento correcto:

```
GHCi> paExample3 = (!! 2)
GHCi> paExample3 "dog"
'g'
```

Lo importante es recordar que los paréntesis no son opcionales.

length

The length function is obvious; it gives you the length of the list!

```
GHCi> length [1..20]
```

```
20
```

```
GHCi> length [(10,20),(1,2),(15,16)]
```

```
3
```

```
GHCi> length "quicksand"
```

```
9
```

reverse

As expected, reverse reverses the list:

```
GHCi> reverse [1,2,3]
```

```
[3,2,1]
```

```
GHCi> reverse "cheese"
```

```
"eseehc"
```

isPalindrome

isPalindrome word = word == reverse word

```
GHCi> isPalindrome "cheese"
```

```
False
```

```
GHCi> isPalindrome "racecar"
```

```
True
```

```
GHCi> isPalindrome [1,2,3]
```

```
False
```

```
GHCi> isPalindome [1,2,1]
```

```
True
```


elem

The elem function takes a value and a list and checks whether the value is in the list:

```
GHCi> elem 13 [0,13 .. 100]
```

```
True
```

```
GHCi> elem 'p' "cheese"
```

```
False
```

respond

```
respond phrase = if '!' `elem` phrase  
then "wow!"
```

```
else "uh.. okay"
```

```
GHCi> respond "hello"
```

```
"uh.. okay"
```

```
GHCi> respond "hello!"
```

```
"wow!"
```

elem es una función que puede tratar como un operador infijo para facilitar la lectura. Cualquier función binaria puede tratarse como un operador infijo encerrándola entre comillas (`). En ejemplo, la función responder devuelve una respuesta diferente dependiendo de si una cadena tiene un signo de exclamación, como sigue.

take and drop

The take function takes a number and a list as arguments and then returns the first n elements of the list:

```
GHCi> take 5 [2,4..100]
```

```
[2,4,6,8,10]
```

```
GHCi> take 3 "wonderful"
```

```
"won"
```

If you ask for more values than a list has, take gives you what it can, with no error:

```
GHCi> take 1000000 [1]
```

```
[1]
```

takeLast

`takeLast n aList = reverse (take n (reverse aList))`

```
GHCi> takeLast 10 [1..100]  
[91,92,93,94,95,96,97,98,99,100]
```

The **drop** function is similar to `take`, except it removes the first `n` elements of a list:

```
GHCi> drop 2 [1,2,3,4,5]  
[3,4,5]  
GHCi> drop 5 "very awesome"  
"awesome"
```

zip

Utilice `zip` cuando desee combinar dos listas en pares de tuplas. Los argumentos de `zip` son dos listas. Si una de las listas es más larga, `zip` se detendrá cuando una de las dos listas, esté vacía:

```
GHCi> zip [1,2,3] [2,4,6]  
[(1,2),(2,4),(3,6)]
```

```
GHCi> zip "dog" "rabbit"  
[('d','r'),('o','a'),('g','b')]
```

```
GHCi> zip ['a' .. 'f'] [1 .. ]  
[('a',1),('b',2),('c',3),('d',4),('e',5),('f',6)]
```

cycle

La función `cycle` es particularmente interesante porque utiliza la evaluación perezosa para crear una lista infinita. Dada una lista, `cycle` la repite infinitamente. Esto puede parecer algo inútil, pero resulta útil en un sorprendente número de situaciones. Por ejemplo, en computación numérica necesitar una lista de n unos. Con `cycle`, esta función es trivial de hacer.

ones

```
ones n = take n (cycle [1])
```

```
GHCi> ones 2
```

```
[1,1]
```

```
GHCi> ones 4
```

```
[1,1,1,1]
```

`cycle` puede ser extremadamente útil para dividir a los miembros de una lista en grupos. Imagine que quiere dividir una lista de archivos y ponerlos en n número de servidores, o del mismo modo repartir los empleados en n equipos. La solución general es crear una nueva función, **assignToGroups**, que toma un número de grupos y una lista, y luego hace un ciclo a través de los grupos, asignando miembros.

assignToGroups

assignToGroups n aList = zip groups aList

where groups = cycle [1..n]

```
GHCi> assignToGroups 3 ["file1.txt","file2.txt","file3.txt"  
                        ,"file4.txt","file5.txt","file6.txt","file7.txt"  
                        ,"file8.txt"]
```

```
[(1,"file1.txt"),(2,"file2.txt"),(3,"file3.txt"),(1,"file4.txt"),  
 ➡ (2,"file5.txt"),(3,"file6.txt"),(1,"file7.txt"),(2,"file8.txt")]
```

```
GHCi> assignToGroups 2 ["Bob","Kathy","Sue","Joan","Jim","Mike"]
```

```
[(1,"Bob"),(2,"Kathy"),(1,"Sue"),(2,"Joan"),(1,"Jim"),(2,"Mike")]
```

Estas funciones son sólo algunas de las más comunes de una amplia gama de funciones de lista que Haskell ofrece. No todas las funciones sobre listas están incluidas en el módulo estándar de Prelude. Todas las funciones sobre listas, incluidas las que se incluyen automáticamente en Prelude, están en el módulo Data.List. Una lista exhaustiva de las funciones Data.List puede encontrarse en línea:

Haskell documentation (<https://hackage.haskell.org/package/basedocs/Data-List.html>).

FUNCIONES DE ORDEN SUPERIOR

Una función de orden superior es técnicamente cualquier función que toma otra función como argumento. Normalmente, cuando se mencionan las funciones de orden superior, un grupo específico de ellas viene a la mente, y casi todas ellas se utilizan para abstraer patrones comunes de recursividad.

¡La verdadera cura para los dolores de cabeza recursivos es la abstracción!

Función Map

Es difícil exagerar la importancia de la función map en la programación funcional y en Haskell. La función map toma otra función y una lista como argumentos y aplica esa función a cada elemento de la lista.

```
GHCi> map reverse ["dog","cat", "moose"] ["god","tac","esoom"]
```

```
GHCi> map head ["dog","cat", "moose"]  
"dcm"
```

```
GHCi> map (take 4) ["pumpkin","pie","peanutbutter"]  
["pump","pie","pean"]
```

FUNCIÓN FOLD(FOLDL Y FOLDR, EQUIVALENTE A REDUCE)

Fold es una función de orden superior porque toma una **función** como uno de sus argumentos, y esta función se aplica repetidamente a los elementos de la lista.

Dependiendo de la dirección en que se aplique) reduce una lista a un solo valor, aplicando una función a los elementos de la lista de manera acumulativa.

foldl :: (a -> b -> a) -> a -> [b] -> a

Primero, toma una función de tipo (a -> b -> a) que se aplica a los elementos de la lista. Luego, toma un valor inicial de tipo a, que se usa como el valor base para la acumulación. Finalmente, toma la lista de elementos de tipo b y la recorre, acumulando el resultado en un solo valor de tipo a

Ejemplo

sumar :: [Int] -> Int

sumar lista = foldl (+) 0 lista

Ejemplo de uso:

sumar [1, 2, 3, 4] -- Resultado: 10

FUNCIÓN FILTER)

Filter es una función de orden superior, muy útil para trabajar con listas. Filter toma una función de predicado (una función que devuelve True o False) y una lista, y devuelve una nueva lista que contiene solo aquellos elementos de la lista original que satisfacen el predicado (es decir, para los cuales la función de predicado devuelve True).

filter :: (a -> Bool) -> [a] -> [a]

(a -> Bool) es una función que toma un valor de tipo a (un elemento de la lista) y devuelve un valor booleano (True o False). [a] es la lista original. La función devuelve una nueva lista que contiene solo los elementos para los cuales el predicado devuelve True.

Ejemplo de uso de filter:

Supongamos que queremos filtrar los números pares de una lista de enteros:

filtrarPares :: [Int] -> [Int]

filtrarPares lista = filter even lista

- **even** es una función predefinida en Haskell que devuelve True si un número es par y False si es impar.
- **filter even lista** aplicará el predicado even a cada elemento de la lista y devolverá una lista solo con los números pares.

EVALUACIÓN DE PATRONES

Haskell evalúa las definiciones de la función en orden. Si un patrón coincide con los argumentos de la función, se ejecuta ese bloque de código. Si no coincide, se sigue buscando otro patrón que se ajuste.

Ejemplo

```
miFuncion :: Int -> String
miFuncion 0 = "Cero"
miFuncion 1 = "Uno"
miFuncion 2 = "Dos"
miFuncion 3 = "Tres"
miFuncion _ = "Otro número"
```

GUARDAS O PROTECCIONES

Haskell, las "guardas" (o protecciones) se utilizan para especificar condiciones adicionales en las definiciones de funciones. Cada guardia se separa con el símbolo | y permite que la función se ejecute solo cuando se cumple una determinada condición.

```
miFuncion :: Int -> String
miFuncion n
  | n == 0  = "Cero"
  | n == 1  = "Uno"
  | n == 2  = "Dos"
  | n == 3  = "Tres"
  | otherwise = "Otro número"
```

En este caso, las guardas se utilizan para evaluar el valor de n con diferentes condiciones. El | indica una guardia, y cada una se evalúa en orden. Si ninguna de las condiciones previas se cumple, se usa la guardia otherwise, que actúa como un caso por defecto, similar al patrón _

Favor calificar la clase

